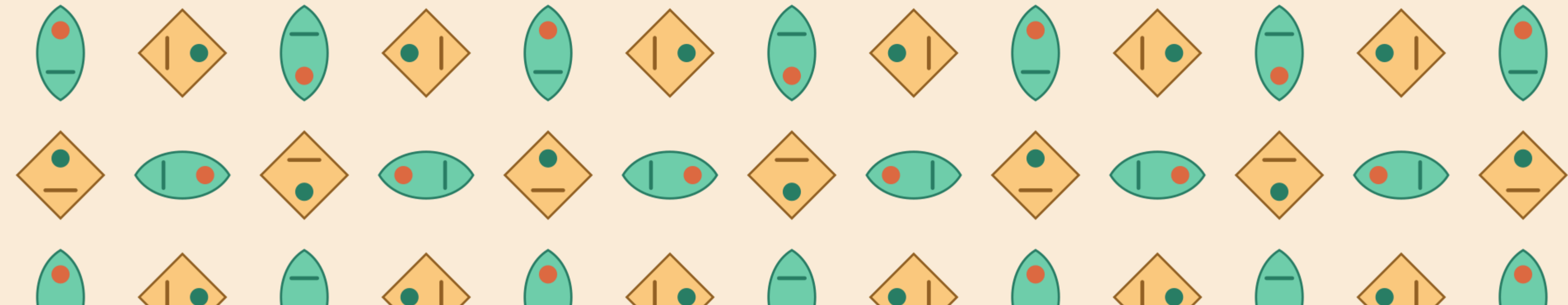

Computer graphics — project proposal

Kitenge

A symmetry-based pattern studio

Design one motif. The program builds the whole pattern.

Janny Jonyo · Kristina Kemoi · Dennis Murage · Neema Mapelu · Cindy Ogutu



One motif in, a whole pattern out

What we are building

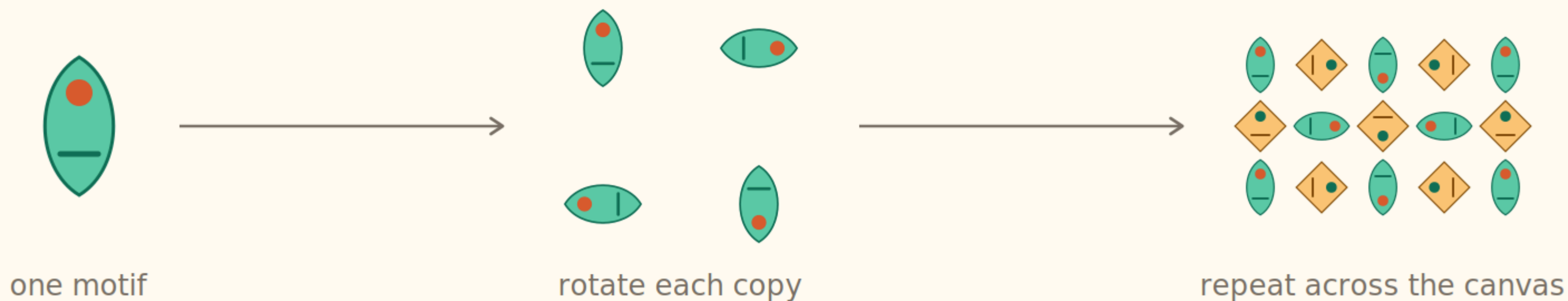
A desktop application that generates repeating geometric patterns from one small shape.

What it does

The user picks a motif, a symmetry rule, colours and tile size. The program draws every copy itself and redraws the canvas instantly.

What the user sees

A control panel beside a live canvas, an animated playback of the pattern assembling, and the pattern wrapped onto a lit 3D surface.



Why it should exist, and who for



The need

Printed fabric, tiling and wallpaper are all built from one motif repeated across a surface. Designing that by hand means copying the motif over and over and hoping the repeat lines up.



Our answer

You design a single tile. The program applies the symmetry rule, places every copy and redraws the whole canvas the moment you change anything.

Who it is for



Design and art students

Test symmetry ideas quickly, without hand-repeating a motif.



Print and textile businesses

Draft patterns for fabric, packaging and signage in minutes.



Students learning geometry

Watch rotation and reflection matrices do visible work on screen.



What we will build, and what we will not

Core — must have

-  Our own pixel rasteriser
-  Six built-in motifs
-  Three symmetry rules: translate, 4-fold, mirror
-  Live canvas with tile size and spacing controls
-  RGB / HSV colour picker and palette
-  Animated assembly playback

Stretch — if time allows

-  Pattern texture-mapped onto a lit 3D surface (OpenGL)
-  Bezier pen tool for drawing your own motif
-  Colour reduction from an uploaded photo

Out of scope

-  Full wallpaper-group support
-  Vector file import or export
-  Freehand drawing
-  Multi-level undo history
-  3D beyond one surface



Every technique, and why we need it

No built-in drawing calls — every pixel is written by an algorithm we implement ourselves.

Technique or algorithm	Why our project needs it
 Bresenham and DDA lines	Straight edges of every motif and the tile guides
 Midpoint circle and ellipse	Round elements inside the motifs
 Polygons and boundary fill	Colouring the enclosed regions of each motif
 Bezier curves	Motifs with curved sides, and the pen tool if we add it
 Line clipping	Trimming the tiles that run past the canvas edge
 2D transformations, homogeneous coordinates	Rotating, mirroring and placing every copy — the core of the project
 Raster operations and double buffering	Flicker-free redraw and the animated assembly
 Colour models: RGB and HSV	Palette picker, and reducing a photo to a five-colour palette
 Texture mapping and lighting	Wrapping the finished pattern onto a lit 3D surface
stretch goal	



How the application will look

An interface mock-up, not a screenshot — the layout is what we are committing to.



Controls on the left

Motif, symmetry rule, tile size, spacing and palette.



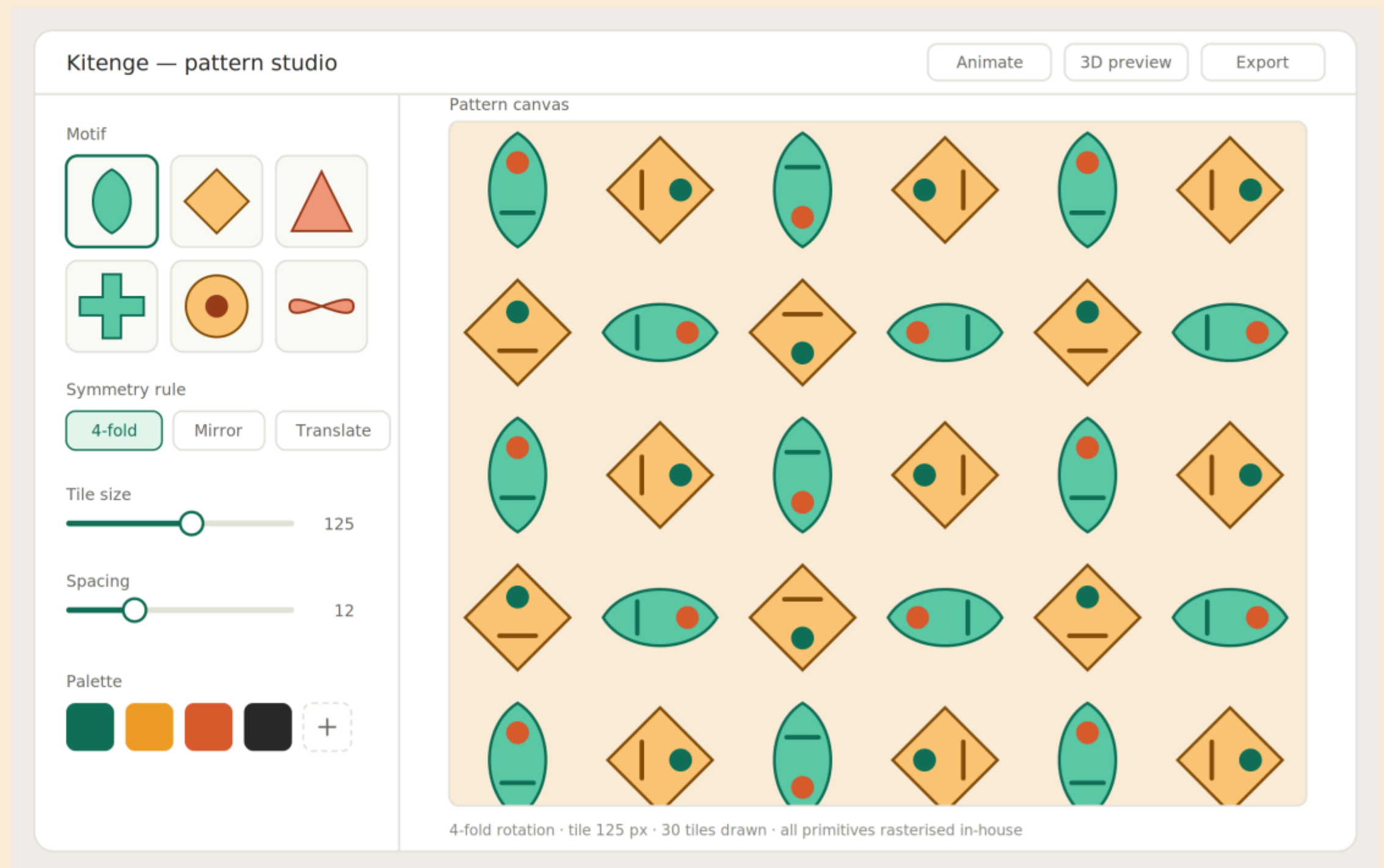
Canvas on the right

Redraws in full every time a control changes.



Status line underneath

Reports the rule, tile size and number of tiles drawn.



Four phases, built week by week

Each phase uses the technique taught in that week's lab, so the project and the coursework are the same work.

Phase 1 · Weeks 1-3

Concept and setup



Finalise the concept
Set up the GitHub repo
Agree the framebuffer
interface everyone draws into

Phase 2 · Weeks 4-5

Core graphics



Lines, circles, polygons
Motif geometry and fill
First shape on screen

Phase 3 · Weeks 7-9

Advanced graphics



Bezier curves
Symmetry engine
Assembly animation

Phase 4 · Weeks 11-12

Integration



OpenGL textured surface
Testing and refinement
Repository documentation



Who owns what

Split by module so five people can work in parallel — everyone commits to the repo weekly.

	Janny Jonyo	Rasteriser and primitives Framebuffer, Bresenham and DDA, midpoint circle and ellipse, line clipping
	Kristina Kemoi	Curves and fill Bezier evaluation, motif geometry, boundary fill and area attributes
	Dennis Murage	Symmetry engine Transformation matrices, tiling logic, the animated assembly
	Neema Mapelu	Interface and colour Layout and controls, RGB / HSV picker, palette handling
	Cindy Ogutu	3D, integration and testing OpenGL surface, merging the modules, test harness, repo documentation

